발표 대본

LangChain 버전 마이그레이션 코드 어시스턴트 · 15분 (본편 약 13분 + 질문) · 발표 [이름]

화면은 훅만
큰 숫자 · 한 문장 · 에러 한 줄. 설명은 말로 합니다.

장마다 세 박자
훅 한 마디 → 설명 → 넘기는 말로 다음 장 예고

멈추기
숫자를 말한 뒤, 질문을 던진 뒤 1초 쉬기

01 질문 한 줄

0:00 ~ 0:40

훅 ChatGPT가 짜 준 코드, 그대로 실행해 보신 적 있으신가요?

안녕하세요, 4팀 발표를 맡은 [이름]입니다.
시작하기 전에 하나 여쭤보겠습니다.
ChatGPT가 짜 준 코드, 그대로 실행해 보신 적 있으신가요?
(잠깐 멈춤)
잘 돌아가면 다행인데요. 라이브러리 코드는 그렇지 않은 경우가 많습니다.
오늘은 LLM이 준 코드를 믿어도 되는지에 대한 이야기입니다.

→ **넘기며** 제가 실제로 받아 본 코드를 보여 드리겠습니다.

02 빨간 에러 한 줄

0:40 ~ 1:30

훅 첫 줄부터 에러입니다.

LLM에게 LangChain 코드를 부탁하면 이런 코드를 줍니다.
최신 LangChain에서 실행하면, 첫 줄부터 에러입니다.
LangChain이 1.0에서 패키지를 나누고, 자주 쓰던 LLMChain 같은 기능을 뺐기 때문입니다.
코드는 맞아 보이는데, 지금은 돌아가지 않는 코드입니다.

→ **넘기며** 이게 저만 겪는 문제일까요?

03 큰 숫자 42%

1:30 ~ 2:30

훅 열 번 중 여섯 번은 실패합니다.

연구 결과를 보면, 문서가 부족할 때 LLM이 만든 코드는 42%만 목표 버전에서 돌아갑니다.
문서를 줘도 66% 정도입니다.
이유는 학습 데이터에 있습니다. 인터넷 코드 대부분이 옛 버전이라, LLM은 옛 방식을 답합니다.
이걸 '버전 지체 환각'이라고 부릅니다.

→ **넘기며** 그런데 진짜 문제는 따로 있습니다.

04 한 문장

2:30 ~ 3:00

훅 LLM은 자기 답이 틀렸다는 걸 모릅니다.

LLM은 자기 답이 틀렸다는 걸 모릅니다.
그래서 틀린 코드를 자신 있게 줍니다.
결국 개발자가 에러를 하나씩 검색하며 고쳐야 합니다.

→ **넘기며** 그래서 저희는 두 가지를 더했습니다.

05 카드 두 장

3:00 ~ 3:50

훅 근거, 그리고 검사. 이 두 가지가 전부입니다.

첫째, 기억이 아니라 근거로 고칩니다. 옛 경로와 새 경로를 잇는 대응표, 그리고 최신 공식 문서를 보고 고칩니다.
둘째, 고친 코드를 검사합니다. 틀리면 이유를 들고 다시 고칩니다.
근거, 그리고 검사. 이 두 가지가 저희 챗봇의 전부입니다.

→ **넘기며** 이 두 가지가 어떻게 이어지는지 보여 드리겠습니다.

06 흐름 그림

3:50 ~ 5:00

훅 검사에 실패하면, 이유를 들고 다시 찾습니다.

흐름은 네 단계입니다.
먼저 바꿀 곳을 찾고, 근거를 찾고, 그 근거로 고치고, 마지막에 검사합니다.
검사에 실패하면 실패 이유를 들고 다시 근거를 찾습니다. 이때 틀렸던 근거는 빼고 찾습니다.
최대 세 번입니다.
이 단계들을 LangGraph로 잇는 일은 1번이 맡았습니다.

→ **넘기며** 그럼 근거는 어디서 올까요?

07 큰 숫자 9,155

5:00 ~ 6:00

훅 옛 경로와 새 경로를 잇는 대응표가 9,155줄입니다.

근거는 2번이 만들었습니다.
옛 import 경로를 새 경로로 잇는 대응표가 9,155줄입니다.
여기에 공식 문서 295개를 잘게 나눠, 필요한 부분을 바로 찾을 수 있게 했습니다.
그리고 바뀐 줄마다 어떤 근거를 썼는지 ID가 붙습니다. 나중에 왜 이렇게 바꿨는지 따라갈 수 있습니다.

→ **넘기며** 근거로 고친 코드, 이제 검사합니다.

08 큰 숫자 11

6:00 ~ 7:00

훅 검사에는 LLM을 쓰지 않습니다.

검사는 3번이 만들었습니다. 모두 11가지인데, LLM을 쓰지 않습니다.
이유는 간단합니다. 검사하는 LLM도 같은 옛 지식을 갖고 있기 때문입니다.
그래서 코드로 확인합니다. 새 경로가 정말 있는지, 근거 없이 바뀐 줄은 없는지, 결과 모양이 그대로인지 봅니다.
근거 없이 바뀐 줄은 곧 환각이라서, 바로 걸러 냅니다.

→ **넘기며** 이 결과를 사용자는 어떻게 볼까요?

09 화면 캡처

7:00 ~ 8:00

훅 코드를 넣고, 말로 부탁하면 됩니다.

제가 맡은 화면입니다.

코드를 넣고 '0.2로 되돌려 줘'처럼 말로 부탁하면 됩니다. 버전을 적지 않으면 최신으로 바꿉니다.

기다리는 동안 지금 어느 단계인지, 검사에서 떨어져 다시 고치는 중인지가 실시간으로 보입니다.

결과가 나오면 지운 줄은 빨강, 새 줄은 초록으로 보이고, 근거와 11가지 검사 결과도 탭으로 볼 수 있습니다.

대화는 TiDB에 저장돼서, 언제든 다시 열어 볼 수 있습니다.

→ **넘기며** 실제로 돌려 본 결과를 보여 드리겠습니다.

10 이전 · 이후 코드

8:00 ~ 9:10

훅 처음 두 번은 떨어지고, 세 번째에 통과했습니다.

어제 실제로 돌려 본 결과입니다.

왼쪽은 예전 방식의 LLMChain 코드입니다. 오른쪽처럼 최신 방식으로 바뀌었습니다.

중요한 건 과정입니다. 처음 두 번은 검사에 걸렸고, 실패 이유를 들고 다시 고쳐서 세 번째에 통과했습니다.

바뀐 여섯 줄 모두 근거가 붙었습니다. 같은 코드를 두 번 돌렸고, 두 번 모두 세 번째에 통과했습니다.

→ **넘기며** 그럼 일반 LLM보다 얼마나 나을까요?

11 큰 숫자 30

9:10 ~ 10:00

훅 같은 질문, 같은 모델, 같은 채점으로 비교했습니다.

일반 LLM과 비교하려고, 실제 사용자가 보낼 법한 질문 30개를 만들었습니다.
요청 문장과 옛 코드가 함께 들어 있고, 열 가지 유형을 세 개씩 담았습니다.
30개 모두 최신 버전에서 실제로 깨지는 코드인지 미리 확인했습니다.
비교는 공정하게 했습니다. 같은 질문, 같은 모델, 같은 채점 기준입니다.

→ **넘기며** 결과입니다.

12 숫자 두 개

10:00 ~ 11:00

훅 [결과 한 문장: 예) 일반 LLM의 [__]배입니다]

[실험 결과가 나오면 채워 주세요]
일반 LLM은 [__]%, 저희 챗봇은 [__]%였습니다.
특히 [__] 유형에서 차이가 가장 컸습니다.
다시 고치는 과정 덕분에, 첫 시도 [__]%에서 [__]%까지 올라갔습니다.

→ **넘기며** 물론 한계도 있습니다.

13 한 문장 + 다음 단계 둘

11:00 ~ 12:10

훅 검사를 통과했다고, 정답은 아닙니다.

한계도 분명히 말씀드리겠습니다.
검사를 통과했다고 정답은 아닙니다. 지금은 코드를 검사할 뿐, 실제로 실행한 결과가 맞는지는 아직 보지 않습니다.
또 검사하고 다시 고치느라, 한 번 바꾸는 데 1분 정도 걸립니다.
다음 단계는 실제 실행까지 확인하는 것, 그리고 변환을 더 빠르게 만드는 것입니다.

→ **넘기며** 마지막으로 정리하겠습니다.

14 마무리 한 문장

12:10 ~ 13:00 (+ 질문)

훅 라이브러리가 바뀌어도, 근거와 함께.

라이브러리는 앞으로도 계속 바뀝니다.
저희 챗봇은 그때마다, 근거와 함께 검사한 코드를 드립니다.
1번부터 4번까지 네 명이 그래프, 데이터, 검증, 화면을 나눠 만들었습니다.
감사합니다. 질문 받겠습니다.

→ **넘기며** (질문이 나오면 부록 1~3장을 꺼내 답하기)

질문이 나오면 (부록)

부록1 ERD

질문용

훅 대화 기록은 이렇게 저장합니다.

users에는 아이디가 중복 없이 들어가고, chat_history에는 질문 하나와 답 하나가 한 행으로 들어갑니다.
새 대화를 누르면 새 session_id가 생기고, 목록은 session_id로 묶어 보여 줍니다.
TiDB에 연결할 수 없으면 화면이 멈추지 않고 로컬 저장으로 넘어갑니다.

부록2 검사 11가지

질문용

훅 검사 항목 전체입니다.

문법, 근거 ID가 이번에 보여 준 근거인지, 기록 없이 바뀐 줄이 없는지, 옛 import가 남았는지, 새 경로가 1.x에 있는지,
옛 메서드, 안 쓰는 import, 반환값 보존, 정의 안 된 이름, 출력 꺼내기, 체인 입출력 키까지 봅니다.
오류는 다시 고칠 이유가 되고, 경고는 결과에 표시만 합니다.

부록3 질문 구성

질문용

훅 평가 질문 구성입니다.

열 가지 유형을 세 개씩, 쉬움 7, 중간 15, 어려움 8입니다.
옛 버전으로 되돌리는 질문 6개까지 36개 모두, 옛 코드가 실제로 깨지는지와 기대하는 답이 실제로 되는지 미리 확인했습니다.

예상 질문 답변

통과하면 정답인가요? 아닙니다. 코드 검사까지이고, 실행 결과 비교는 다음 단계입니다. (13장)

왜 검사를 LLM에게 맡기지 않았나요? 검사하는 LLM도 같은 옛 지식을 갖고 있어서, 코드로 확인했습니다. (부록 2)

근거를 지어낼 수 있지 않나요? 이번에 실제로 보여 준 근거 ID만 인정하고, 기록 없이 바뀐 줄은 걸러 냅니다.

30개로 충분한가요? 열 가지 유형 · 난이도별 경향을 보는 게 목적입니다. 다음엔 실사용 코드로 늘립니다. (부록 3)

왜 1분이나 걸리나요? 검사에 실패하면 최대 세 번 다시 고치기 때문입니다. 속도 개선이 다음 단계입니다.

대화는 어디에 저장되나요? TiDB에 사용자 · 대화 두 테이블로 저장합니다. (부록 1)